
Facultad de Informática - Universidad Nacional de La Plata

Segment Tree

Autor: Luciano Agustín Macías

Materia: Taller de Tecnologías de Producción de Software (O7301) - Opción Técnicas y Estrategias para la resolución de problemas

Contenido

Contenido.....	2
Introducción.....	2
Propiedades de las operaciones soportadas.....	3
Ejemplo de operaciones válidas.....	4
Estructura y Representación.....	4
Construcción (build).....	5
Actualización (set).....	6
Consulta (query).....	8
Extensión y Mejoras.....	10
Resolución de Problemas.....	11
A. Segment Tree for the Sum.....	11
C. Number of Minimums on a Segment.....	14
Observación.....	16
Aplicaciones.....	17
Conclusión.....	17
Material Consultado.....	17

Introducción

Cuando nos enfrentamos a problemas donde tenemos que actualizar o consultar en un intervalo dado dentro de un rango de valores (típicamente almacenados en un arreglo) por una operación específica como: buscar el mínimo, máximo, la suma, etc.

Se vio durante el transcurso de la cursada del Taller de Tecnologías de Producción de Software que hay diferentes acercamientos según la complejidad del problema a resolver:

- **Por medio de fuerza bruta** → Ante cada consulta se procesa el rango pedido con la operación que corresponda, esto en el peor de los casos requiere que se calcule el rango entero, por lo que, el tiempo de consulta es de $O(n)$. Hacer una actualización puntual solo requiere acceder a la posición específica en el arreglo y modificar el valor, esto lo hace de $O(1)$. Como nota adicional no tiene tiempo de construcción.
- **Emplear algoritmos de prefijos** → Se precalcula la operación desde el comienzo del arreglo hasta la posición correspondiente, por ejemplo en la suma la posición i del arreglo de prefijos almacena el valor $\sum_{j=0}^i a_j$, esta construcción es de $O(n)$ ya que se accede a cada elemento una única vez para realizar la suma acumulada y luego se utiliza esa suma para el siguiente elemento. Las consultas se responden en tiempo constante $O(1)$ ya que se obtiene la diferencia entre las posiciones $[R]$ y $[L - 1]$. Las actualizaciones requieren un poco más de procesamiento ya que actualizar una posición obliga a recalcular en el peor de los casos todo el arreglo de prefijos, es decir, las actualizaciones son $O(n)$.

- **Prefijos con “lazy” update** → Si el problema a resolver nos asegura que primero recibimos todas las actualizaciones (puntuales o en rango) a la vez, podemos registrar donde realizar dichas modificaciones y hacerlas todas juntas antes o durante la construcción del arreglo de prefijos. Esto termina generando una única actualización de $O(n)$.

Entonces:

Técnica	Build	Query	Update	Space
Arreglo Normal	-	$O(n)$	$O(1)$	$O(n)$
Prefijos	$O(n)$	$O(1)$	$O(n)$	$O(n)$

En esta comparativa se nota de forma clara que cada acercamiento sacrifica una característica para mejorar otra pero en la realidad los problemas de rango dinámico necesitan de técnicas que se adapten mejor y no afecten de forma negativa el tiempo de resolución. Es entonces donde aparecen estrategias que equilibran el tiempo de construcción de la estructura, su actualización y la consulta sobre los intervalos del rango, siendo el protagonista de esta monografía el Segment Tree.

Esta estructura de datos presenta características interesantes que la convierten en el mejor candidato para resolver casi cualquier problema de rango dinámico ya que aprovecha la estructura de los árboles binarios para ofrecer tiempos de respuesta muy eficientes a la vez que permite reutilizar el mismo código con solo un cambio, adaptándolo para cualquier escenario mientras que la operación cumpla ciertas características matemáticas.

Propiedades de las operaciones soportadas

Dado que el algoritmo del Segment Tree realiza de manera arbitraria las operaciones entre los valores, es necesario que la operación definida sea asociativa, es decir, vale que para todo $a, b, c \in \text{Intervalo} \rightarrow (a \otimes b) \otimes c = a \otimes (b \otimes c)$.

También para más simplicidad es deseable que la operación tenga un elemento neutro e tal que para cualquier $a \in \text{Intervalo}$ se cumple que $a \otimes e = a$. En caso de no tener elemento neutro se puede implementar igualmente pero se debe tener especial cuidado con los casos nulos o vacíos al operar.

En cuanto a la propiedad conmutativa tampoco es obligatoria, sin embargo, el orden en que se operan los valores tiene un sentido estricto al combinar respuestas de los subárboles, por ejemplo, en concatenación de strings no es lo mismo $op(query(izq), query(der))$ que $op(query(der), query(izq))$.

Lo descrito hasta este punto es el concepto de **Monoide** (o Monoide Conmutativo si cumple conmutatividad).

- Operación Asociativa $\rightarrow (a \otimes b) \otimes c = a \otimes (b \otimes c)$

- Neutro $\rightarrow a \otimes e = a$
- Conmutativa $\rightarrow a \otimes b = b \otimes a$

Ejemplo de operaciones válidas

Operación	Neutro
Mínimo	+infinito
Mínimo + Ocurrencias	(+infinito, 0)
Máximo	-infinito
Máximo + Ocurrencias	(-infinito, 0)
Suma	0
Multiplicación	1
XOR	00...0
AND	11...1
OR	00...0
GCD	0

Entre otras se encuentran módulo multiplication, matrix multiplication, bitwise operations: $\&$, $|$, $\wedge$, el máximo común divisor (en este caso, el cálculo del GCD no es constante por lo que la operación toma más tiempo).

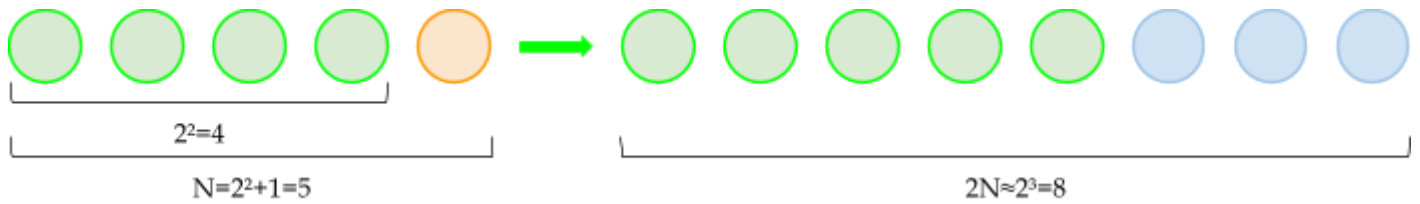
Estructura y Representación

Un Segment Tree es un árbol binario donde cada nodo representa el resultado de la operación en un rango dado del arreglo original.

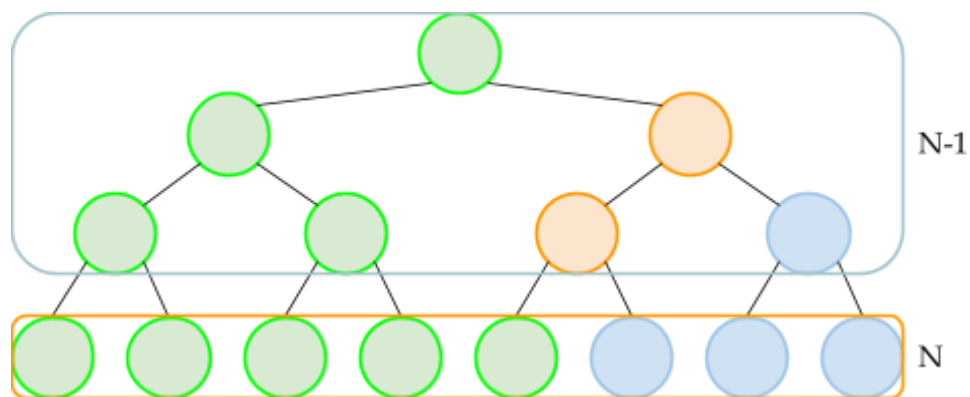
- La raíz representa la operación aplicada al arreglo completo.
- Los nodos internos representan el resultado de la operación sobre un fragmento del arreglo original
- Las hojas son los elementos originales.

Para mantener las propiedades de índices simplificados al representar el árbol con un arreglo ($HI = raíz * 2$; $HD = raíz * 2 + 1$) y la eficiencia general de la estructura, es necesario que el arreglo tenga un tamaño igual a una potencia de 2 para construir un árbol perfecto (completamente lleno hasta el último nivel). En caso de que el arreglo no tenga tamaño igual a una potencia de 2, se podría completar con un elemento neutro para la operación hasta llegar a un tamaño igual a la potencia de 2 más próxima. En el peor escenario el arreglo mide uno más que la potencia de 2 más cercana siendo necesario

entonces, prácticamente duplicar el tamaño, pasando de un arreglo de tamaño N a uno de aproximadamente $2N$.



Por propiedad de los árboles binarios perfectos, si tenemos N hojas, tenemos $N-1$ nodos internos, entonces en el peor caso terminamos teniendo $2N$ hojas y $2N-1$ nodos internos lo que nos lleva a $4N-1$ o aproximadamente $4N$ nodos o celdas en el arreglo que representa el árbol.



El algoritmo se compone fundamentalmente de 4 operaciones:

- $op(v1, v2) \rightarrow$ Realiza la operación definida entre dos valores dados.
- $build(idx, l, r) \rightarrow$ Construye el árbol haciendo uso de la operación.
- $set(idx, val) \rightarrow$ Modifica la posición idx del array con el valor val .
- $query(l, r) \rightarrow$ responde a la operación dada en el rango $[L ; R - 1]$.

Construcción (build)

Usando la operación definida se construye el árbol de abajo hacia arriba.

Como se mencionó, los nodos hoja contienen directamente los valores del arreglo original y luego, cada nodo interno se calcula aplicando op sobre los valores de sus dos hijos, de esta manera cada nodo representa la combinación de un segmento del arreglo, hasta llegar a la raíz, que contiene el resultado correspondiente a todo el rango. Este algoritmo se implementa de la siguiente forma:

```
build(node, l, r):  
    if l == r:  
        tree[node] = arr[l]
```

```

    return
    mid = (l + r) / 2

    build(node*2, l, mid)
    build(node*2 + 1, mid + 1, r)

    tree[node] = op(tree[node*2], tree[node*2 + 1])

```

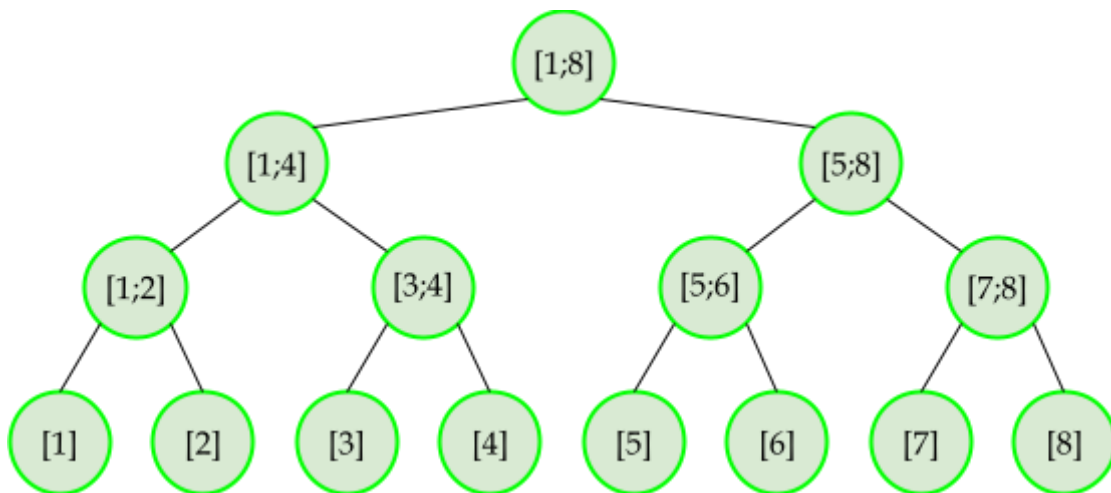
Donde:

- Node → es el índice que representa al nodo en el arreglo del árbol.
- L y R → son los índices del arreglo original que corresponden al rango que representa el nodo dado.

Si L y R son iguales quiere decir que el algoritmo alcanzó una hoja por lo que almacena en ese nodo el valor del arreglo correspondiente a esa posición.

En caso de que L y R no sean iguales, es decir, el algoritmo se encuentra en un nodo interno, es necesario calcular el punto medio para poder continuar la ejecución hacia los hijos, siendo el hijo izquierdo el intervalo $[L ; mid]$ y el hijo derecho $[mid + 1 ; R]$. Eventualmente la ejecución retorna y se puede calcular el valor del nodo actual a partir del valor de sus hijos que ya se encuentran calculados.

Visualmente esta asignación por rangos se ve de la siguiente forma:



Para los N elementos del arreglo original se genera un camino desde la raíz hasta la hoja que corresponde a ese elemento, dicho camino tiene un tamaño de $\log n$, por lo que la complejidad temporal de la construcción es del orden de $O(n \log n)$ debido a que cada camino raíz hoja se construye en $O(\log n)$ y esto se repite N veces.

Actualización (set)

La actualización requiere no solo que se actualice el valor del arreglo original o la hoja correspondiente sino que se recalculen el camino que vincula dicha hoja con la raíz.

Se comienza descendiendo hasta la hoja correspondiente al elemento a modificar. Una vez alcanzado y actualizado, la recursión al empezar a retornar, recalcula los nodos internos aplicando nuevamente la operación definida sobre los nodos del camino.

El método `set` expone una firma sencilla que recibe el índice del arreglo y el valor a establecer en esa posición. Dentro se utiliza un método con una firma óptima para recursión en árboles.

```
set(idx, val): update(1, 1, n, idx, val)

update(node, l, r, idx, val):
    if l == r:
        tree[node] = val
        return

    mid = (l + r) / 2

    if idx <= mid:
        update(node*2, l, mid, idx, val)
    else:
        update(node*2 + 1, mid+1, r, idx, val)

    tree[node] = op(tree[node*2], tree[node*2 + 1])
```

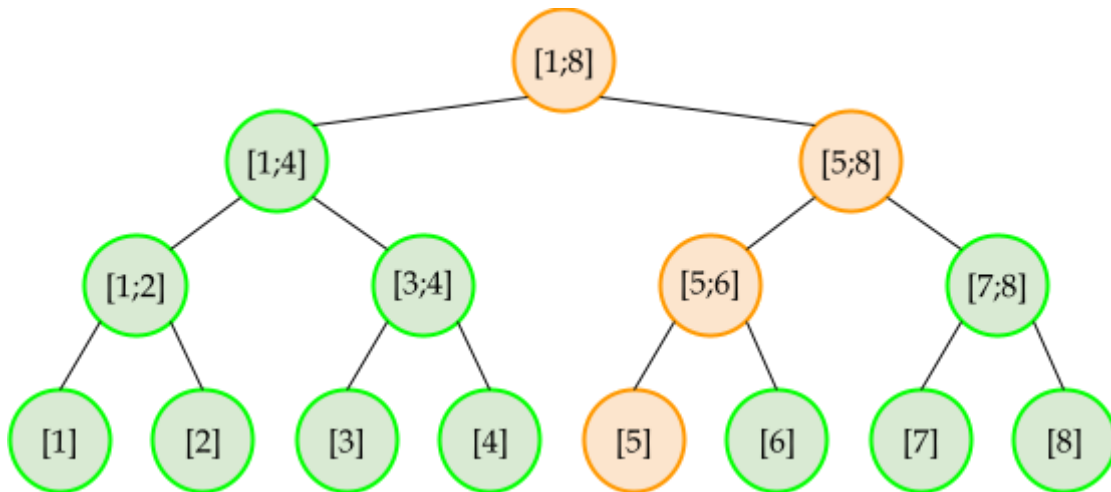
Donde:

- Node → es el índice que representa al nodo en el arreglo del árbol.
- L y R → son los índices del arreglo original que corresponden al rango que representa el nodo dado.
- Idx → índice del arreglo original donde se quiere insertar el valor.
- Val → valor a insertar en la posición dada.

Este algoritmo se parece en gran medida al de construcción, sin embargo, sólo desciende el árbol por el camino que lleva a la hoja específica que se tiene que modificar. Al retornar se calcula nuevamente con la operación dada sólo los nodos que componen el camino desde la raíz hasta la hoja modificada.

Como la altura del árbol corresponde al valor $\log n$, una actualización puntual con este algoritmo tiene una complejidad temporal $O(\log n)$.

Supongamos que se actualiza la posición 5 del arreglo, los nodos naranja son los nodos afectados:



Consulta (query)

La idea principal consiste en recorrer únicamente los segmentos relevantes para el rango consultado.

En cada nodo visitado puede ocurrir uno de tres casos posibles:

1. El segmento está completamente fuera del rango buscado.
2. El segmento está completamente contenido en el rango.
3. El segmento se superpone parcialmente con el rango.

Si nos encontramos en el primer caso, se debe detener la recursión del algoritmo en ese punto y retornar el valor neutro que define nuestra operación.

Cuando un segmento está totalmente contenido, se retorna directamente la información almacenada en ese nodo.

Por último, si existe una superposición parcial se debe continuar explorando los hijos para combinar a través de la operación definida en op los valores que estos retornan.

El método query expone una firma sencilla que recibe el rango a consultar. Dentro se utiliza un método con una firma óptima para recursión en árboles.

```
query(l, r):  
    return get(1, 1, n, l, r)
```

```
get(node, sl, sr, l, r):  
    if sr < l or r < sl:  
        return 0  
  
    if l <= sl and sr <= r:  
        return tree[node]
```

```
mid = (sl + sr) / 2

left  = get(node*2,    sl,    mid, l, r)
right = get(node*2 + 1, mid+1, sr,  l, r)

return op(left, right)
```

Donde:

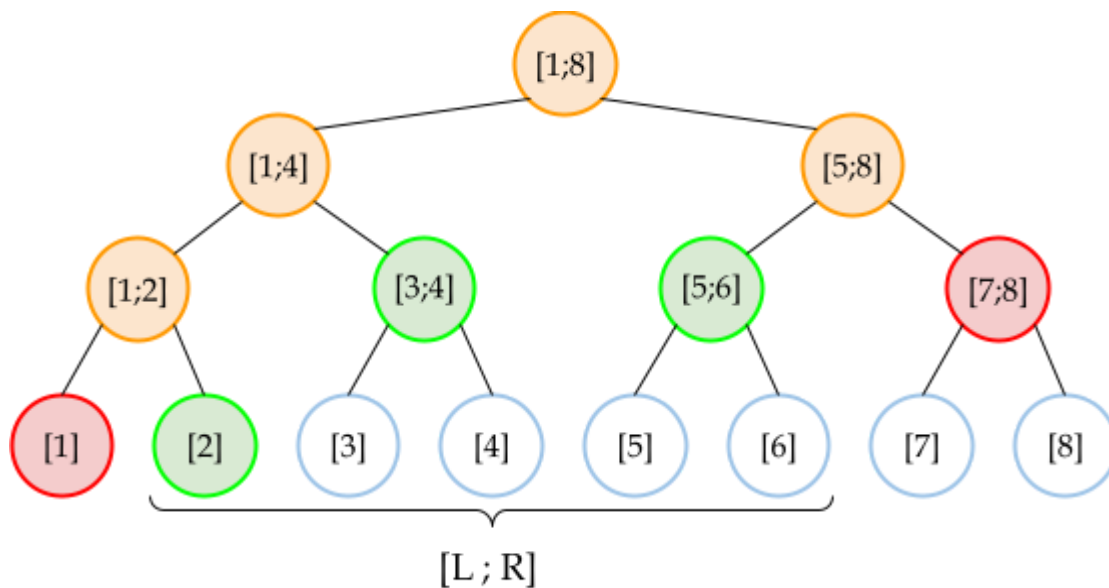
- Node → es el índice que representa al nodo en el arreglo del árbol.
- SL y SR → son los índices del arreglo original que corresponden al rango que representa el nodo dado.
- L y R → índices del arreglo original que corresponden al rango que se está consultando.

El método get implementa los casos en el orden que se definió antes:

1. Para comprobar que se está fuera del rango basta con saber si el extremo superior del nodo está por debajo del extremo inferior de consulta ($sr < l$) o si el extremo superior de consulta está por debajo del extremo inferior del nodo ($r < sl$).
2. Si el extremo inferior de consulta es menor o igual al extremo inferior del nodo ($l \leq sl$) y además el extremo superior del nodo es menor o igual al extremo superior de consulta ($sr \leq r$), se toma el valor completo de ese nodo ya que está completamente contenido.
3. Por último si está parcialmente contenido se invoca de forma recursiva obteniendo el resultado de los hijos y luego combinándolos con la operación definida.

Es importante remarcar que el 0 del return 0 en realidad representa el neutro de la operación como ya se mencionó en las propiedades de las operaciones soportadas.

Si ilustramos cómo actúa este algoritmo obtendremos un gráfico como el siguiente:



Los nodos pintados son por todos los que pasó el algoritmo y si diferenciamos por colores:

- **Rojos** → La ejecución se detiene ya que el nodo está fuera del rango
- **Naranjas** → La ejecución continua ya que el rango del nodo se superpone parcialmente con el rango buscado
- **Verdes** → Estos nodos se encuentran contenidos por completo dentro del rango y son los que se combinan para responder la consulta.

En el peor caso, la query necesita tomar segmentos tanto del extremo derecho del subárbol izquierdo como del extremo izquierdo del subárbol derecho. Esto obliga a recorrer ambos caminos hasta las hojas respectivas, visitando a lo sumo una cantidad proporcional a $2 \log(n)$ nodos. lo que sigue siendo un orden de $O(\log n)$

Extensión y Mejoras

- **Actualización en rango** → Consiste en aplicar una modificación a todos los elementos dentro de un intervalo, por ejemplo sumar un valor x a cada posición del rango. En un Segment Tree tradicional, esto significa bajar hasta cada hoja afectada, actualizar su valor y después recalculer todos los nodos internos correspondientes hasta la raíz. En el peor de los casos, una actualización podría afectar a todos los elementos y ya que cada actualización individual requiere $O(\log n)$ operaciones resulta en una complejidad total de: $O(n \log n)$. Existen algoritmos más eficientes que disminuyen esa complejidad hasta un orden de $O(n)$
- **Actualización diferida (Lazy Propagation)** → Permite hacer actualizaciones sobre rangos de manera eficiente evitando propagar inmediatamente los cambios a todos los nodos afectados ya que en lugar de actualizar cada segmento bajando hasta las hojas, el árbol marca ciertos nodos como “pendientes de actualización”. De esta forma, la propagación real del cambio solo pasa cuando una query o una nueva

actualización necesita acceder a ese segmento. Gracias a esta estrategia, se evita trabajo innecesario sobre regiones del árbol que podrían no volver a consultarse, reduciendo significativamente el costo de múltiples actualizaciones de rango. Matemáticamente, esta estrategia transforma una operación que originalmente costaría $O(n)$ en una de tiempo logarítmico $O(\log n)$. Para conseguir esto, la estructura requiere de un arreglo secundario (comúnmente llamado lazy) de tamaño $4N$, dedicado exclusivamente a almacenar los valores de actualización pendientes antes de ser transferidos a los nodos hijos.

Resolución de Problemas

A. Segment Tree for the Sum

Ref: [A. Segment Tree for the Sum](#)

En esta tarea se pide realizar un segment tree para la suma.

Input

La primera línea tiene dos enteros n y m ($1 \leq n$ y $m \leq 100000$), siendo n el tamaño del array y m la cantidad de operaciones. La siguiente línea tiene n números que representan el estado inicial del arreglo, cada número puede estar entre $0 \leq a_i \leq 10^9$. Las siguientes m líneas describen las operaciones a realizar, siendo:

- $1\ i\ v$: modifica el elemento en el índice i por el valor v ($0 \leq i \leq n$, $0 \leq v \leq 10^9$).
- $2\ l\ r$: calcula la suma de los elementos entre los índices l y $r-1$ ($0 \leq l < r \leq n$).

Output

Por cada operación del segundo tipo informar la suma correspondiente.

Código C++

Partiendo del pseudo código definido en los apartados anteriores, el código resulta casi en un 1 a 1 con instrucciones en C++.

```
#include <bits/stdc++.h>
using namespace std;
#define FASTIO ios::sync_with_stdio(false); cin.tie(nullptr);
#define ll long long
struct SegTree
{
    vector<ll> tree;
    vector<ll> arr;
    SegTree(vector<ll> original_arr)
        : tree(original_arr.size() * 4),
```

```

        arr(original_arr)
    {
        build(1, 0, arr.size() - 1);
    }
    ll op(ll a, ll b)
    {
        return a + b;
    }
    void build(ll node, ll l, ll r)
    {
        if (l == r)
        {
            tree[node] = arr[l];
            return;
        }
        ll mid = (l + r) / 2;
        build(node * 2, l, mid);
        build(node * 2 + 1, mid + 1, r);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    void set(ll idx, ll val)
    {
        update(1, 0, arr.size() - 1, idx, val);
    }
    void update(ll node, ll l, ll r, ll idx, ll val)
    {
        if (l == r)
        {
            tree[node] = val;
            return;
        }
        ll mid = (l + r) / 2;
        if (idx <= mid)
            update(node * 2, l, mid, idx, val);
        else
            update(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
    }
    ll query(ll l, ll r)
    {
        return get(1, 0, arr.size() - 1, l, r);
    }

```

```

}
ll get(ll node, ll sl, ll sr, ll l, ll r)
{
    if (sr < l || r < sl)
        return 0;
    if (l <= sl && sr <= r)
        return tree[node];
    ll mid = (sl + sr) / 2;
    ll left = get(node * 2, sl, mid, l, r);
    ll right = get(node * 2 + 1, mid + 1, sr, l, r);
    return op(left, right);
}
};
int main()
{
    FASTIO
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n);
    for (ll i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    SegTree st(a);
    ll comando, p1, p2; // p1= idx o l; p2= val o r
    while(m--){
        cin >> comando >> p1 >> p2;
        if (comando == 1) {
            st.set(p1, p2);
        }else{
            cout << st.query(p1, p2-1) << '\n';
        }
    }
    return 0;
}

```

Submissions

#	When	Who	Problem	Lang	Verdict	Time	Memory
375660397	May/22/2026 12:04 ^{UTC-3}	lordvi	A - Segment Tree for the Sum	C++23 (GCC 14-64, msys2)	Accepted	140 ms	1800 KB

C. Number of Minimums on a Segment

Ref: [C. Number of Minimums on a Segment](#)

Se pide cambiar el código para no solo retornar el mínimo sino también qué cantidad de veces aparece el mínimo en el rango.

Input

La primera línea tiene dos enteros n y m ($1 \leq n$ y $m \leq 100000$), siendo n el tamaño del array y m la cantidad de operaciones. La siguiente línea tiene n números que representan el estado inicial del arreglo, cada número puede estar entre $0 \leq a_i \leq 10^9$. Las siguientes m líneas describen las operaciones a realizar, siendo:

- 1 i v : modifica el elemento en el índice i por el valor v ($0 \leq i \leq n$, $0 \leq v \leq 10^9$).
- 2 l r : calcular el mínimo y el número de elementos iguales al mínimo de los elementos entre los índices l y $r-1$ ($0 \leq l < r \leq n$).

Output

Por cada operación del segundo tipo informar dos enteros: el mínimo del segmento y el número de elementos iguales al mínimo.

Código C++

Partiendo del pseudo código definido en los apartados anteriores, el código resulta casi en un 1 a 1 con instrucciones en C++.

```
#include <bits/stdc++.h>
using namespace std;
#define FASTIO ios::sync_with_stdio(false); cin.tie(nullptr);
#define ll long long
const ll INF = 1000000000LL;
struct Minimo
{
    ll minimo = INF;
    ll ocurrencias = 0;
    Minimo(ll min = INF, ll ocu = 0)
        : minimo(min), ocurrencias(ocu){}
};
struct SegTree
{
    vector<Minimo> tree;
    vector<ll> arr;
    SegTree(vector<ll> original_arr)
        : tree(original_arr.size() * 4),
          arr(original_arr)
    {}
};
```

```

{
    build(1, 0, arr.size() - 1);
}
Minimo op(Minimo a, Minimo b)
{
    if(a.minimo==b.minimo){
        return Minimo(a.minimo, a.ocurrencias+b.ocurrencias);
    }
    return (a.minimo<b.minimo) ? a : b;
}
void build(ll node, ll l, ll r)
{
    if (l == r)
    {
        tree[node] = Minimo(arr[l], 1);
        return;
    }
    ll mid = (l + r) / 2;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
}
void set(ll idx, ll val)
{
    update(1, 0, arr.size() - 1, idx, val);
}
void update(ll node, ll l, ll r, ll idx, ll val)
{
    if (l == r)
    {
        tree[node] = Minimo(val, 1);
        return;
    }
    ll mid = (l + r) / 2;
    if (idx <= mid)
        update(node * 2, l, mid, idx, val);
    else
        update(node * 2 + 1, mid + 1, r, idx, val);
    tree[node] = op(tree[node * 2], tree[node * 2 + 1]);
}
Minimo query(ll l, ll r)
{
    return get(1, 0, arr.size() - 1, l, r);
}
Minimo get(ll node, ll sl, ll sr, ll l, ll r)
{

```

```

        if (sr < l || r < sl)
            return Minimo();
        if (l <= sl && sr <= r)
            return tree[node];
        ll mid = (sl + sr) / 2;
        Minimo left = get(node * 2, sl, mid, l, r);
        Minimo right = get(node * 2 + 1, mid + 1, sr, l, r);
        return op(left, right);
    }
};

int main()
{
    FASTIO
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n);
    for (ll i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    SegTree st(a);
    ll comando, p1, p2; // p1= idx o l; p2= val o r
    while(m--){
        cin >> comando >> p1 >> p2;
        if (comando == 1) {
            st.set(p1, p2);
        }else{
            Minimo m = st.query(p1, p2-1);
            cout << m.minimo << ' ' << m.ocurrencias << '\n';
        }
    }
    return 0;
}

```

Submissions 							
#	When	Who	Problem	Lang	Verdict	Time	Memory
375662699	May/22/2026 12:25 UTC-3	lordvi	C - Number of Minimums on a Segment	C++23 (GCC 14-64, msys2)	Accepted	140 ms	5000 KB

Observación

Si analizamos detenidamente ambos códigos, podemos notar que son prácticamente idénticos con la única diferencia que solo cambia el tipo de dato que almacena el árbol y la operación definida.

Aplicaciones

Más allá de la programación competitiva, la estructura Segment Tree tiene muchas aplicaciones interesantes entre las que se destacan:

- **Renderizado de Gráficos** → Algoritmos que ofrecen mejora de rendimiento en escenas interactivas con trazado de rayos, donde se utiliza un segment tree para controlar ciertos parámetros de los rayos de forma eficiente. Ref: [Interactive Ray-Traced Scene Editing Using Ray Segment Trees | Springer Nature Link](#)
- **Geometría computacional y GIS** → Problema de la ventana para un conjunto de segmentos de línea paralelos a los ejes, es un problema clásico de búsqueda geométrica. Ref: [Computational Geometry - Algorithms and Applications](#) (pag 231)

Conclusión

La estructura Segment Tree que se desarrolló en este informe demuestra ser una herramienta poderosa no solo dentro del ámbito de la programación competitiva, sino que también lo es en aplicaciones del mundo real. Se basa en fundamentos matemáticos sólidos que permiten resolver una amplia variedad de problemas con una eficiencia destacable, equilibrando las operaciones que la componen y permitiendo reutilizar casi la totalidad de su código con modificaciones simples. Gracias a ser una solución robusta, para problemas simples o que no requieran de tantas actualizaciones se convierte en complejidad adicional cuando existen otras alternativas que se adaptan mejor a esos casos.

Por lo tanto, si bien es la “navaja suiza de los problemas de rango dinámico”, hay que saber elegir cuándo y dónde utilizarlo, aún más cuando el tamaño de la memoria es limitado ya que su complejidad espacial es de orden $O(4n)$, no es un detalle para tomar a la ligera si tenemos demasiados elementos.

Material Consultado

- [ITMO Academy - Segment Tree](#)
- Anotaciones personales del taller de programación competitiva ICPC (UNLP)